CORE103L Computational Thinking
LW8 - Lists



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
1.1 Instructions 2
1.2 Marking scheme 2
1.2.1 Lab Objectives 2
1.3 Late submission policy 2
1.4 Use of AI 2
1.5 Viva 3

2 Lab Exercises **4**
2.1 Reading Integers into a List 4
2.2 Sum of Numbers 5
2.3 Wanna (reverse a) piece of me? 6
2.4 Muhammad Huzaifa XI 7
2.5 Noisy Neighbors 9
2.6 Hindu Shuffle 10

Introduction

1.1 Instructions

- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *LW8_aa123456.zip* where *aa123456* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise of both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 10.

1.2.1 Lab Objectives

Following are the lab objectives of this lab:

- Master the fundamentals of Python lists, including creation, indexing, and manipulation.
- Apply lists to solve programming challenges, demonstrating proficiency in list-based data manipulation and problem-solving.

1.3 Late submission policy

There is no late submission policy for the lab.

1.4 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.5 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Lab Exercises

2.1 Reading Integers into a List

This question should be implemented in the file q1.py.

Problem Description

Write a function named **read_integer_list** that takes an integer count as a parameter. The function should:

1. Create an empty list.
2. Use a loop to read count integer inputs from the user, one at a time.
3. Store each integer in the list.
4. Return the list of integers.

Note

count will always be a non-negative integer.

Sample Interaction

In the sample below, lines written as [Input -> value] represent values entered by the user while **read_integer_list** is running. They are **not** output produced by the function.

```
>>> read_integer_list(3)
[Input -> 5]
[Input -> 10]
[Input -> 15]
[5, 10, 15]
```

Testing

In order to test your function, type the following command on the terminal:

```
python -m pytest tests/test_q1.py
python3 -m pytest tests/test_q1.py
```

2.2 Sum of Numbers

This question should be implemented in the file q2.py.

Function Descriptions

You are required to implement the following functions:

1. Write a function named **sum_lst** that takes a list lst as a parameter. The function should **sum all the items** in the list and **return the sum**.
2. Write a function named **main** that takes no parameters. The function should:
   - Take a string input that contains **space-separated integers**.
   - Convert the string into a list of integers and store it in the variable lst.
   - Call **sum_lst**, passing lst as a parameter.
   - Print the output returned by the **sum_lst** function.

Note

The input will contain one or more integers. The integers may be positive, negative, or zero.

Sample Interaction

In the sample below, lines written as [Input -> value] represent values entered by the user while the function is running. They are **not** output produced by the function.

```
>>> main()
[Input -> 10 20 30 40 50]
150
```

Sample Interaction Explanation

The **main()** function reads the input and converts it into a list of integers:

lst = [10, 20, 30, 40, 50]

The **main()** function then calls **sum_lst** and prints the sum returned by the function.

Testing

In order to test your function, type the following command on the terminal:

```
python -m pytest tests/test_q2.py
python3 -m pytest tests/test_q2.py
```

2.3 Wanna (reverse a) piece of me?

This question should be implemented in the file q3.py.

Problem Description

In Python, lists have a built-in method called reverse() that reverses all elements in a list in place. This means the original list is changed directly.

Write a function called **reverse_slice** that takes a list t and two integer indices, start and stop, as arguments. The function should reverse all elements in the list slice t[start:stop] **in place**. The function should not return a value. If the selected slice is empty, list t should remain unchanged.

Note

As with Python slicing, the start index is included while the stop index is excluded. Indices may be zero, positive, or negative, and may also indicate an empty slice. Negative indices count from the end of the list. Your solution must handle all such situations. The order of elements outside the selected slice must remain unchanged.

There are no restrictions for this question. Feel free to use list slicing and built-in functions.

Assigning a List to a Slice

Python allows a list slice to be replaced with another list using **slice assignment**. This modifies the original list **in place**; it does not create a new list.

```
>>> lst = [0, 1, 2, 3, 4, 5]
>>> replacement = [9, 8]
>>> lst[2:4] = replacement
>>> lst
[0, 1, 9, 8, 4, 5]

>>> lst = [0, 1, 2, 3, 4, 5]
>>> replacement = [9, 8]
>>> lst[-3:-1] = replacement
>>> lst
[0, 1, 2, 9, 8, 5]
```

Sample Interaction

```
>>> t = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> reverse_slice(t, 3, 7)
>>> t
[0, 1, 2, 6, 5, 4, 3, 7, 8, 9]
```

```
>>> t = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> reverse_slice(t, 2, -4)
>>> t
[0, 1, 5, 4, 3, 2, 6, 7, 8, 9]

>>> t = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> reverse_slice(t, 0, 10)
>>> t
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

>>> t = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> reverse_slice(t, 0, -1)
>>> t
[8, 7, 6, 5, 4, 3, 2, 1, 0, 9]

>>> t = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> reverse_slice(t, 20, 30)
>>> t
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

>>> t = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> reverse_slice(t, -20, -10)
>>> t
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Testing

In order to test your function, type the following command on the terminal:

```
python -m pytest tests/test_q3.py
python3 -m pytest tests/test_q3.py
```

2.4 Muhammad Huzaifa XI

This question should be implemented in the file q4.py.



Problem Description

Muhammad Huzaifa is preparing a shortlist of players for his team *Guru Gorillas* before selecting the final Playing XI for the Habib Sports League.

Before making the final selection, he needs to remove some **excluded players** from the registration list. The registration list may also contain an empty string.

According to the team's selection rules, a player is considered excluded if:

- the player's name does **not contain any vowel** (`a, e, i, o, u`), or
- the player's name is `"babarazam"`, ignoring letter case.

Function Description

Write a function **remove_excluded_players** that takes a list player_names containing player names and removes all excluded players.

- The function must modify the original list **in place**.
- The function should **not return anything**.
- The order of the remaining player names must be preserved.
- Vowel checking and the comparison with `"babarazam"` should both be case-insensitive.

Sample Interaction

```
>>> player_names = [
    "quddusi", "MNT", "swifthyderabad", "CEmajorAT",
    "skibidiali", "valverde", "baqir", "ronaldo",
    "hassan", "alonso", "etoo", "mrgreedy",
    "mbappe", "messi", "evra", "mraura",
    "amir", "mufaddal", "isco", "saifibhai",
    "iniesta", "abdullah", "babarazam",
    "hussainshah", "sr4", "kdb", "askarizz",
    "basim", "raphina", "usman", "coldp@lm4r", ""
    ]

>>> remove_excluded_players(player_names)

>>> player_names
['quddusi', 'swifthyderabad', 'CEmajorAT', 'skibidiali',
'valverde', 'baqir', 'ronaldo', 'hassan', 'alonso',
'etoo', 'mrgreedy', 'mbappe', 'messi', 'evra',
'mraura', 'amir', 'mufaddal', 'isco', 'saifibhai',
'iniesta', 'abdullah', 'hussainshah', 'askarizz',
'basim', 'raphina', 'usman', 'coldp@lm4r']
```

Testing

In order to test your function, type the following command on the terminal:

```
python -m pytest tests/test_q4.py
python3 -m pytest tests/test_q4.py
```

2.5 Noisy Neighbors

This question should be implemented in the file q5.py.

Problem Description

Netiquette is the unofficial set of rules for polite online communication. One common rule is that people should not **SHOUT**, that is, write words in all capital letters.

A string is considered **noisy** if all of its alphabetic letters are uppercase. Given a list of strings, find all neighbors of noisy strings.

Function Description

Write a function **noisy_neighbors** that takes a list words and returns a list containing the neighbors of noisy strings.

- A neighbor is the string immediately before or immediately after a noisy string.
- A noisy string may also be a neighbor of another noisy string.
- Each position in the original list should be included only once.
- If the same string occurs at different positions, each occurrence is treated separately.
- The neighbors must be returned in the same order as they appear in the original list.

Note

words may be empty. Each string in words will contain at least one alphabetic letter.

Sample Interaction

```
>>> noisy_neighbors([
    "how", "much", "wood", "would", "a",
    "WOODCHUCK", "CHUCK", "if", "a",
    "woodchuck", "could", "CHUCK", "WOOD"])

['a', 'WOODCHUCK', 'CHUCK', 'if', 'could', 'CHUCK', 'WOOD']

>>> noisy_neighbors([
    "HOW", "much", "WOOD", "would", "a",
    "woodchuck", "chuck", "if", "a",
    "woodchuck", "could", "chuck", "wood"])

['much', 'would']
```

Sample Interaction Explanation

In the first case, `"WOODCHUCK"`, `"CHUCK"`, `"CHUCK"` and `"WOOD"` are noisy. The first `"WOODCHUCK"` and `"CHUCK"` are also neighbors of each other, so both appear in the returned list. The neighboring strings are returned in the same order as they appear in the original list. The two occurrences of `"CHUCK"` are at different positions, so both can appear in the returned list.

In the second case, `"HOW"` and `"WOOD"` are noisy. `"much"` at the same position is a neighbor of both, so that position is included only once.

Testing

In order to test your function, type the following command on the terminal:

```
python -m pytest tests/test_q5.py
python3 -m pytest tests/test_q5.py
```

2.6 Hindu Shuffle

This question should be implemented in the file q6.py.

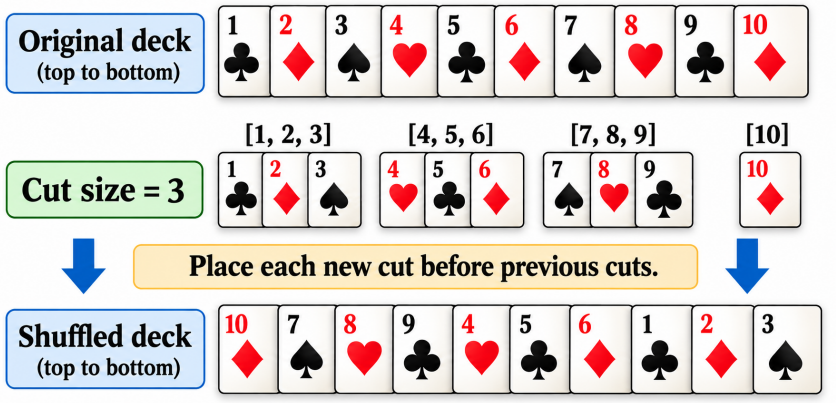
Background

A deck of cards can be shuffled by repeatedly cutting a fixed number of cards from the top of the deck. For this exercise, a **Hindu shuffle** is defined by dividing the deck into consecutive cuts of the same size and placing each new cut before the previous cuts. The final cut may contain fewer cards than the given cut size. The order of items within each cut remains unchanged.

For example, consider the deck:
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
with a cut size of 3.

It is divided into: [1, 2, 3], [4, 5, 6], [7, 8, 9], and [10].

Placing the cuts in reverse order produces: [10, 7, 8, 9, 4, 5, 6, 1, 2, 3].



Function Description

Write a function **hindu_shuffle** that takes two parameters: deck, a list of items, and cut_size, a positive integer.

The function should return a **new list** containing the items of deck arranged according to the shuffle described above. The original list should remain unchanged.

Notes and Constraints

- deck may be empty.
- cut_size will always be a positive integer.
- The final cut may contain fewer than cut_size items.
- The order of items within each cut must be preserved.
- If cut_size is greater than or equal to the length of deck, the returned list will have the same order as deck.

Sample Interaction

```
>>> hindu_shuffle(
    ['aardvark', 'buffalo', 'crocodile', 'dingo',
     'elephant', 'fox', 'gorilla', 'hippopotamus'], 4)
['elephant', 'fox', 'gorilla', 'hippopotamus',
 'aardvark', 'buffalo', 'crocodile', 'dingo']
```

Sample Interaction Explanation

The deck is divided into two cuts of four items:
['aardvark', 'buffalo', 'crocodile', 'dingo']
and
['elephant', 'fox', 'gorilla', 'hippopotamus'].

The second cut is placed before the first cut while the order of items within each cut remains unchanged.

Hint

List slicing can be useful in your solution.

Testing

In order to test your function, type the following command on the terminal:

```
python -m pytest tests/test_q6.py
python3 -m pytest tests/test_q6.py
```